CS 301 | LABORATORY ACTIVITY 7

Implementing Form Controls in the Final Project

Project: Medical and Dental Clinic Management System for the Commission on Audit

Team: De Celis, Kurt Renzo E.; Manila, Chelsey C.; Marcellana, Marc Matthew M.; Rodenas, Chad Andrew A.

Delivered: A standalone, runnable demonstration with two major and two supporting forms; client-side and server-side validation; data handling through Python/SQLite; and rendered form-layout previews.

Important boundary: The demo is not merged into the team's planned Angular / ASP.NET Core codebase, does not use real COA accounts, and must not be used to process actual clinical or payment information.

Form category	Implemented form	Controls
Major 1	Employee-linked Dependent Registration	10
Major 2	Dental Treatment Appointment Request	10 groups / 11 controls counting both radio buttons
Supporting 1	Clinic Record Search & Filter	10
Supporting 2	Employee Contact Profile Update	10

What the user must still supply

The instructor requires actual screenshots from the running app, a 2–4 minute live demo recording, and an assigned GitHub repository with real commits from every group member. Static previews in this package are not substitutes for those items. Use the included demo guide and capture helper on your computer.

1. Technology and data handling

The demonstration uses native HTML inputs, semantic field labels, accessible browser controls and JavaScript client validation. A Python standard-library HTTP server receives JSON API requests, independently validates them, and stores approved demonstration records in an SQLite database.

Layer	Files / responsibilities
User interface	static/index.html and styles.css: four forms, explicit labels, help text, keyboard navigation, visible errors and actions.
Client processing	static/app.js: check required fields, valid formats, dates, file size; prevent submission; show errors and database-driven results.
Server processing	server.py: API endpoints /api/dependents, /api/appointments, /api/search, and /api/profile, each with backend validation.
Storage	data/coa_demo.sqlite3: fictional linked dependent records, appointments and employee contact profile. Created automatically on first run.

2. Form purpose and implemented controls

Major 1: Dependent Registration

Link qualified dependents under an authorized employee account (up to four).

Employee ID (read-only text); Dependent full name (text); Relationship (select); Date of birth (date); Eligibility review status (select); Contact email (email); Mobile number (telephone); Eligibility document (file); Contact address (textarea); Eligibility confirmation (checkbox).

Major 2: Dental Appointment Request

Request dental treatment for linked dependents; staff must confirm the request.

Registered dependent (select); Treatment type (select); Preferred date (date); Preferred time (select); Preferred dentist (select); Contact mobile (telephone); Notification channel (radio group: Email or SMS); Reason for visit (textarea); Special assistance (checkbox); Acknowledgement of staff confirmation (checkbox).

Supporting 1: Clinic Record Search

Filter saved fictional dependents and dental appointments.

Search keywords (search input); Record type (select); Beneficiary category (select); Record status (select); Provider (select); Date from (date); Date to (date); Sort order (select); Include pending (checkbox); Exact full-name match (checkbox).

Supporting 2: Employee Profile Update

Save edited fictional employee contact and emergency-contact information.

Employee ID (read-only text); Display name (text); Email address (email); Mobile number (telephone); Department (text); Office location (text); Preferred contact channel (select); Emergency contact name (text); Emergency mobile (telephone); Accessibility / communication notes (textarea).

3. Validation rules and anticipated behavior

Form	Validation / prevented errors	Effect on valid submission
Dependent registration	Required fields, email and mobile formats, date range, relationship status, four-dependent cap, file extension, actual file signature and 1 MB maximum.	The linked dependent is saved in SQLite for clinic eligibility review.
Dental appointment request	Linked dependent, valid treatment/clinician/time, future date within 90 days, contact number, reason, acknowledgement, conflicting confirmed slot.	Saved as Pending clinic confirmation, not as an automatically confirmed appointment.
Record search	Valid optional dates and date-range ordering; filter values are applied on the server.	Filtered records display in a results table and activity log.
Profile update	Required name/email/mobile/department/office/emergency fields, valid mobile formats, contact method.	SQLite profile is updated; updated values appear on refresh.

4. Accessibility and usability

• Every visible input has an associated text label (or fieldset legend for the radio group).

• Text placeholders and small helper messages explain expected values and upload restrictions.

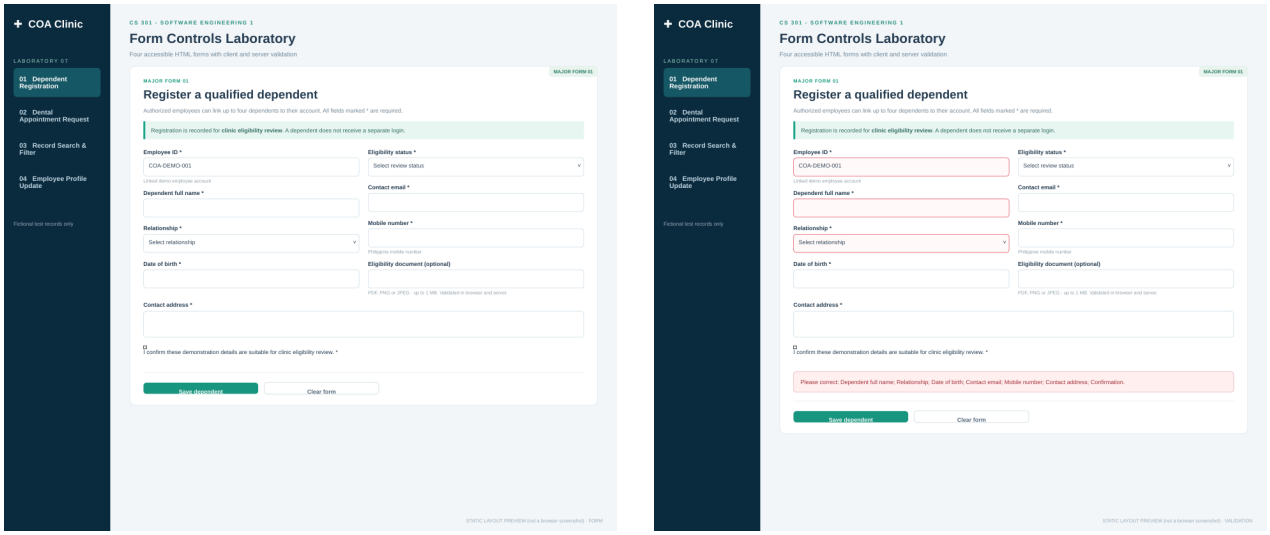
• The page has a skip link, logical navigation order, focus-visible styling and keyboard-operable controls.

• Invalid submissions are blocked; the form identifies invalid controls, gives a visible alert, and moves focus to the first invalid field.

• Buttons use explicit action labels, and success/error messages are exposed through accessible status regions.

5.1 Layout previews — Dependent Registration

The previews below are rendered directly from the form HTML. They illustrate layout and validation messaging, but are not screenshots of a live browser session. Capture live states on your own computer for the instructor.

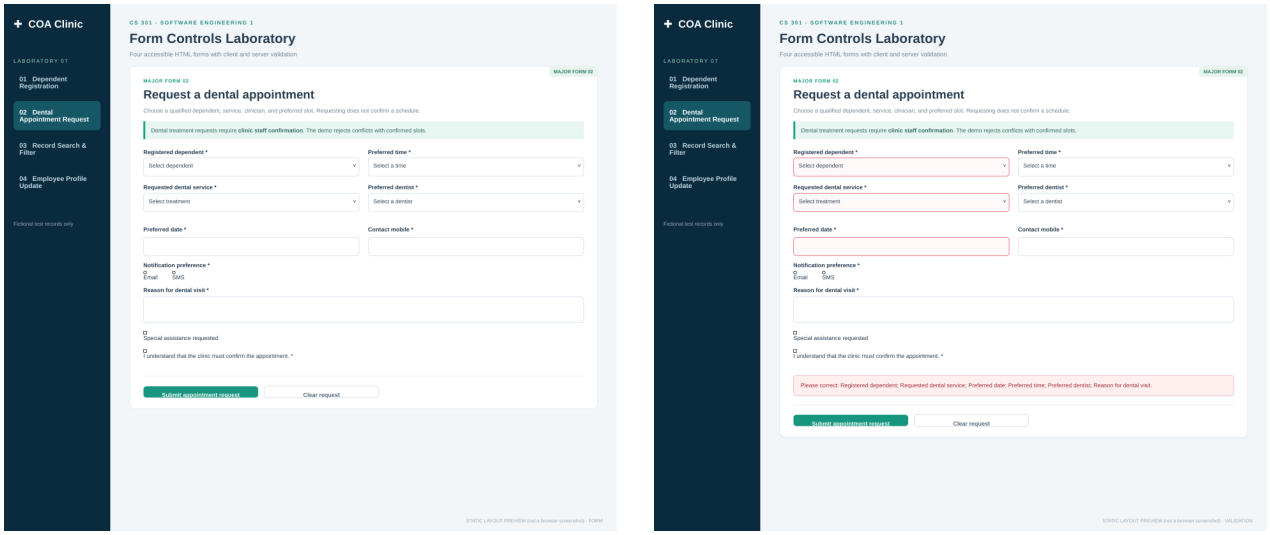


Left: initial form layout. Right: illustrative required-field or validation-error state.

The backend enforces the four-dependent limit and only links records to the demo employee. A proof-file input is validated; only file metadata is retained in this prototype.

5.2 Layout previews — Dental Appointment Request

The previews below are rendered directly from the form HTML. They illustrate layout and validation messaging, but are not screenshots of a live browser session. Capture live states on your own computer for the instructor.

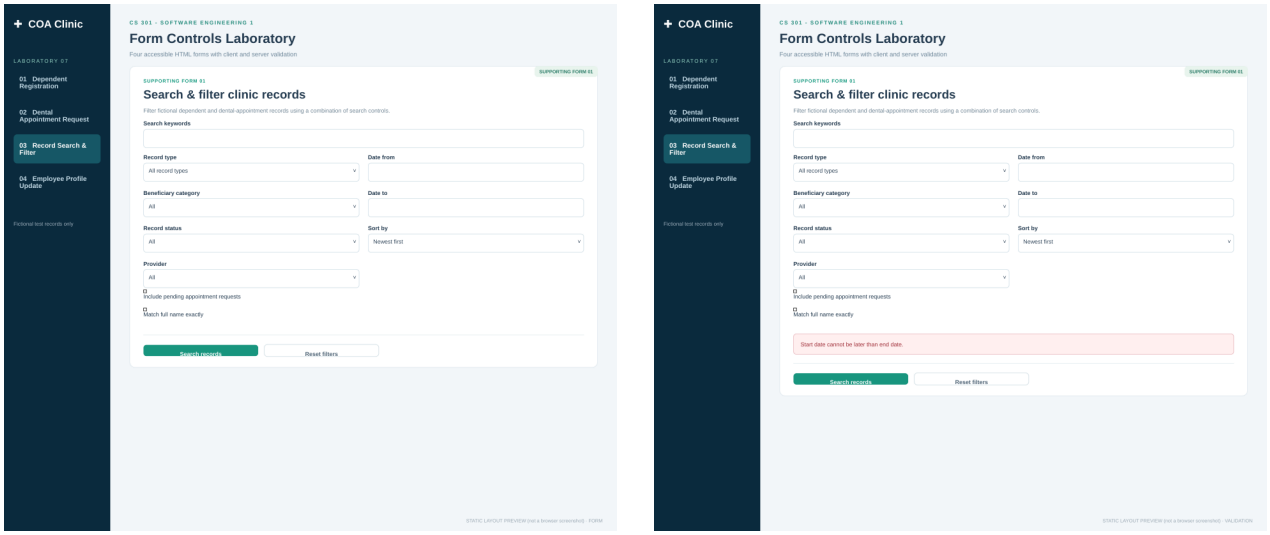


Left: initial form layout. Right: illustrative required-field or validation-error state.

The backend rejects a slot already confirmed for a selected dentist and saves valid requests as pending clinic confirmation.

5.3 Layout previews — Clinic Record Search

The previews below are rendered directly from the form HTML. They illustrate layout and validation messaging, but are not screenshots of a live browser session. Capture live states on your own computer for the instructor.

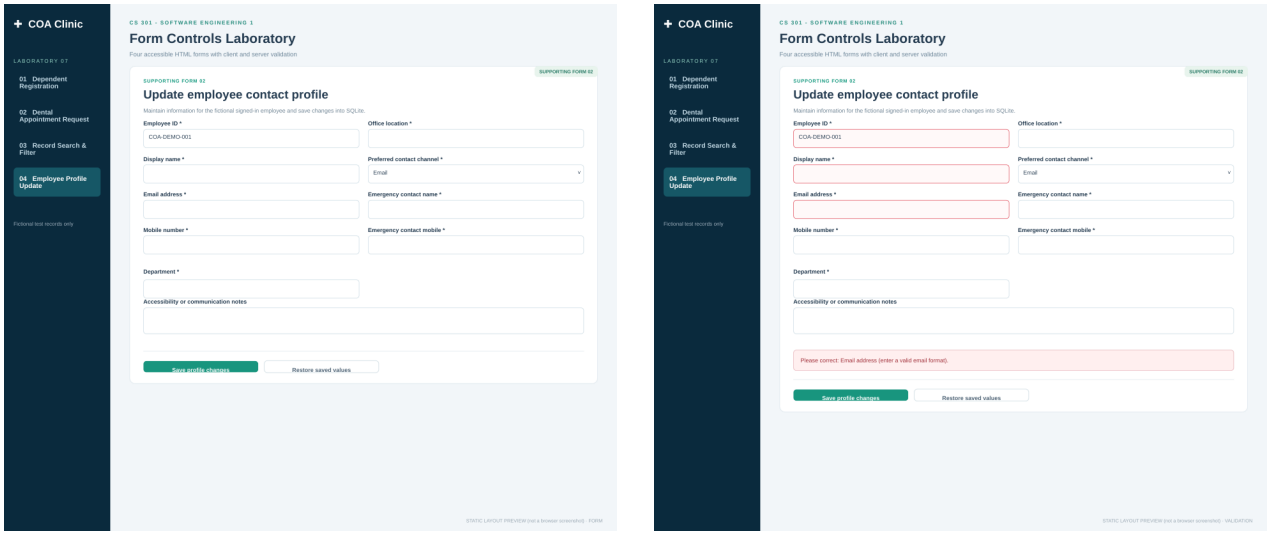


Left: initial form layout. Right: illustrative required-field or validation-error state.

Search inputs are sent to the server. Optional filters control which sample records appear in the results table.

5.4 Layout previews — Employee Profile Update

The previews below are rendered directly from the form HTML. They illustrate layout and validation messaging, but are not screenshots of a live browser session. Capture live states on your own computer for the instructor.



Left: initial form layout. Right: illustrative required-field or validation-error state.

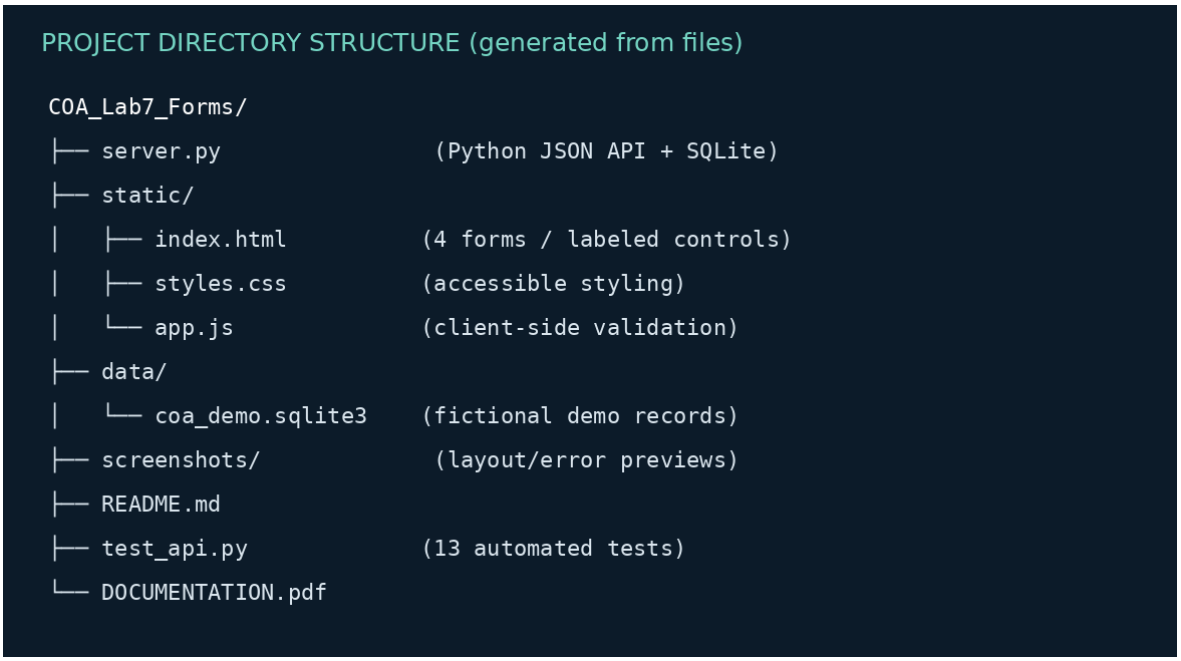
Invalid email and telephone fields are rejected. A valid profile change persists to the fictional employee record in SQLite.

6. Actual backend verification

13 automated Python tests passed. The test suite checks: seeded fictional data, valid/invalid dependent registration, max-four restriction, invalid phone, pending appointment status, slot conflicts, unacknowledged request, record searching, impossible date ranges, profile edits, bad email, and a forged file signature.

To rerun: **python -m unittest -v test_api.py** from the project directory. Browser layout previews are static renders and do not replace live browser tests.

7. Screenshot of project directory structure



Directory listing generated from the actual submitted source files.

8. How to run and capture real submission evidence

1. Run python server.py (Python 3.9 or newer).
2. Open http://127.0.0.1:8765 in Chrome, Firefox or Edge.
3. Capture all four visible forms and each validation state using your browser’s screenshot tool.
4. Record an actual 2–4 minute browser session following DEMO_VIDEO_GUIDE.md.
5. Push the actual team contributions to your assigned GitHub repository. Supply the real URL and each member’s commits.

A standalone prototype is not a substitute for integration into the team’s final Angular/ASP.NET Core project. Include integration evidence only if the group has actually completed it.